Flight Sequencer

Dual-Core RTOS Firmware Design

Embedded Firmware Design Assessment: architecture, synchronization strategy, timing analysis and pseudocode for a launch-vehicle Flight Sequencer on a multi-core MCU running a preemptive RTOS.

Design at a glance

Aspect	Decision
Core split	Core 1 is a dedicated hard-real-time core (Flight Sequencer only). Core 0 handles all asynchronous I/O, supervision and the 200 ms task.
Sensor RX	DMA into ring buffers plus a frame-end (idle-line) interrupt; validation deferred to a high-priority ingest task.
Output TX	DMA transmit with polled completion (timing is under our control, so no ISR is needed).
IPC	Direct task notifications for events; single-writer seqlock latest-value mailboxes for all cross-core and cross-rate data. No mutexes on the hard path.
Feedback loop	16-byte feedback latched by the sequencer only on a fixed 20-frame boundary, so behaviour is repeatable.
Margins	Sequencer worst-case response about 3.9 ms against a 10 ms deadline; Core 0 load about 23%, Core 1 load 30%.

Contents

0 Assumptions | 1 Architecture and task design | 2 Synchronization and concurrency | 3 Pseudocode | 4 Evaluation-criteria traceability

0. Assumptions

The brief leaves hardware details open, so the following engineering assumptions are made explicit. Each can be revised without changing the structure of the design.

#	Assumption
A1	Dual-core 32-bit MCU (200 MHz or faster), shared on-chip SRAM, hardware atomics (LDREX/STREX), a shared free-running 1 µs timebase readable by both cores, per-interrupt core routing (GIC/NVIC target) and a hardware watchdog.
A2	Shared RAM is mapped non-cacheable through the MPU (or is hardware-coherent). Every shared object is 64-byte aligned to avoid false sharing.
A3	RTOS is SMP, preemptive, fixed-priority (FreeRTOS-SMP style, higher number = higher priority) with hard per-task core affinity, 1 kHz tick and static allocation only (no heap after init).
A4	RS422 ports are UARTs at 1 Mbaud, 8N1 (10 µs per byte). Each has its own DMA channel (circular RX or one-shot TX) and idle-line detection.
A5	Every frame = 2 B sync + 1 B sequence + 1 B length + payload + 2 B CRC16 (6 B overhead). Wire times: Sensor 1 (38 B) 0.38 ms, Sensor 2 (60 B) 0.60 ms, actuator (26 B) 0.26 ms, telemetry (64 B) 0.64 ms.
A6	Sensors push frames autonomously at exact periods; their phase relative to the MCU is unknown. The design works for any phase.
A7	WCET budgets, to be confirmed by measurement and static analysis: ISR 5 µs, sensor ingest 40 µs per frame, Flight Sequencer 3 ms, Supervisor 200 µs, Slow task 40 ms.
A8	Failure policy: cyclic tasks never block waiting for data. Stale or invalid input means hold the last good value, raise a flag and enter degraded mode.
A9	The 200 ms feedback is applied only at a fixed frame boundary (every 20th frame), so behaviour is repeatable no matter when the slow task finishes inside its window.

1. System Architecture and Task Design

1.1 Task, timer and ISR definition

ID	Name	Kind	Core	Priority	Trigger / period	Budget
I1	UART_S1_ISR	ISR (idle-line)	0	IRQ lvl 1	Sensor 1 frame end, 5 ms	5 µs
I2	UART_S2_ISR	ISR (idle-line)	0	IRQ lvl 1	Sensor 2 frame end, 10 ms	5 µs
I3	TMR_A_ISR	HW timer ISR	1	IRQ lvl 2	10 ms, phase 0	3 µs
I4	TMR_B_ISR	HW timer ISR	0	IRQ lvl 2	10 ms, phase +6 ms	3 µs
T1	SensorIngestTask	Event task	0	5	Notified by I1 / I2	40 µs per frame
T2	FlightSeqTask	Periodic task	1	5 (highest app task on core)	10 ms, released by I3	3 ms
T3	SupervisorTask	Periodic task	0	4	10 ms, released by I4	200 µs
T4	SlowCompTask	Periodic task	0	2	200 ms, released by I4 every 20th frame	40 ms
-	Idle tasks	Kernel	0, 1	0	-	-

All interrupt priorities are numerically at or below the kernel syscall ceiling so the FromISR APIs are legal. The actuator and telemetry transmit paths use DMA and need no ISR. The hardware watchdog is kicked only by T3.

1.2 Multi-core strategy

- **Core 1 is the deterministic compute core.** It runs only the Flight Sequencer and its timer ISR. No UART interrupt, no logging and no lower-priority task can run there, so jitter comes only from a 3 µs timer ISR and shared-bus contention.
- **Core 0 is the I/O and housekeeping core.** It absorbs all asynchronous events (sensor frame interrupts), ingest, supervision and the slow computation. Its load is about 23%, and the slow task is lowest priority, so everything else preempts it and it can never delay ingestion.
- **Contention is minimised** because every shared object has exactly one writer and a wait-free write path. The cores communicate only through four small mailboxes. There are no cross-core mutexes, queues or kernel calls on the critical path.
- **Why the slow task is on Core 0:** placing a 40 ms task on Core 1 would let it interfere with the hard 10 ms task and would put the cross-core boundary in the wrong place. Core 1 keeps spare capacity for growth.

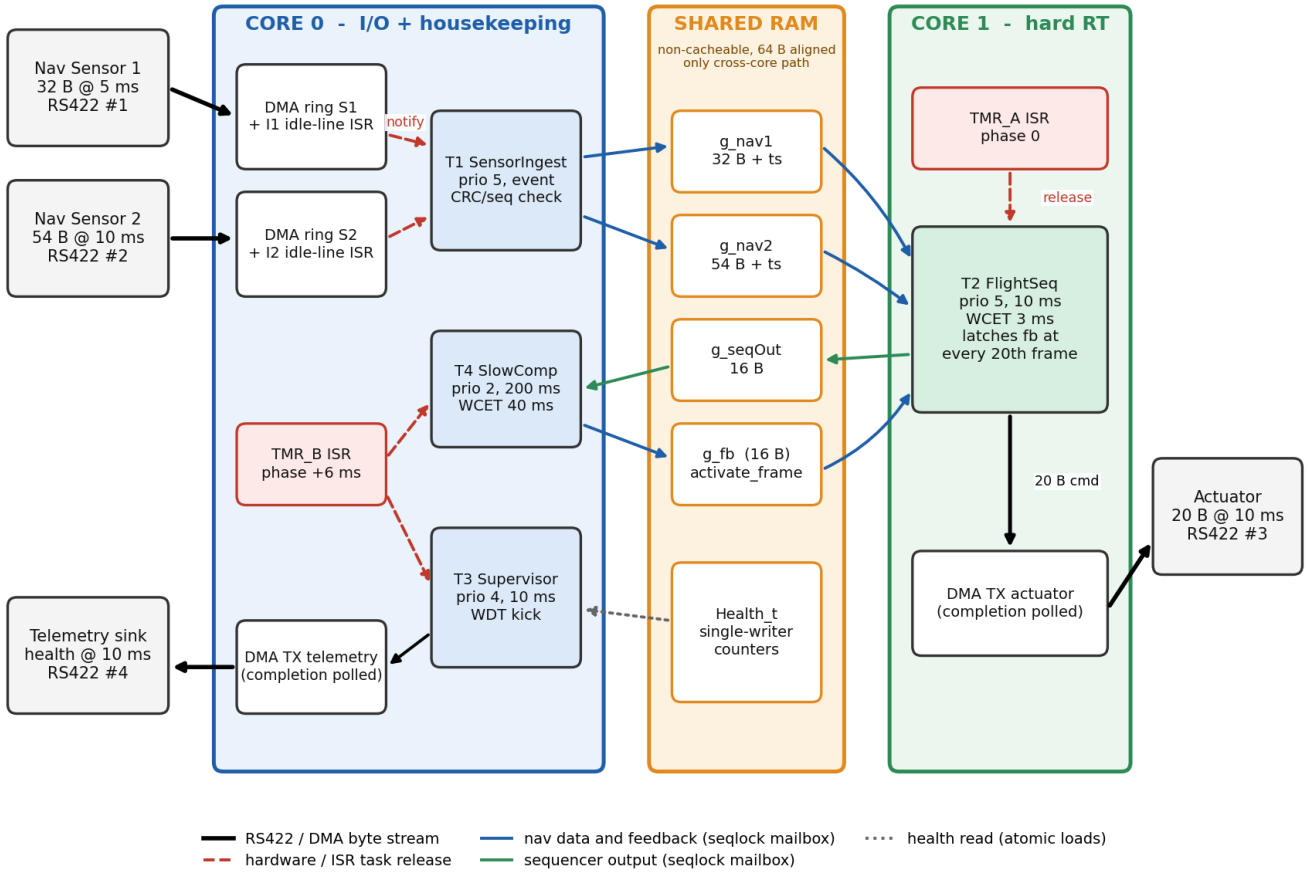


Figure 1. System architecture: cores, ISRs, tasks, DMA rings, the shared-RAM mailboxes and the RS422 ports.

1.3 Data flow and IPC

Data / event	From to	Mechanism	Why
Frame received	UART ISR to T1	Direct task notification	Lightest unblock primitive, same core, no allocation.
Nav 1 / Nav 2 latest samples	T1 (Core 0) to T2 (Core 1) and T4	Seqlock latest-value mailbox	State data: only the newest value matters. A queue would grow stale or drop, and add cross-core locking.
Frame release	TMR_A_ISR to T2; TMR_B_ISR to T3 and T4	Counting direct task notification	Hardware-driven release with no tick jitter. The count also exposes missed releases.
16 B sequencer output	T2 to T4	Seqlock mailbox g_seqOut	Wait-free for the 10 ms writer.
16 B feedback	T4 to T2	Seqlock mailbox g_fb with activate_frame	The reader (T2) can never block on a slow task.
20 B actuator command	T2 to port	DMA TX from a T2-owned buffer	Single owner, nothing to share.
Health data	T1, T2, T4, ISRs to T3	Single-writer atomic counters and timestamps	No locks; one cache line per writer.
Rare events (fault log entries)	T1 to T3 (both Core 0)	Small queue, non-blocking send	Aperiodic and same core only, so safe. Never on a hard path.

1.4 Hardware interface strategy: interrupts vs polling

Option	Pros	Cons for this timing budget
Per-byte RX interrupt	Simple, no DMA.	130 bytes per 10 ms means about 13,000 ISRs per second at about 1.5 µs each (2% CPU) and a preemption every 10 µs during each 0.4 to 0.6 ms burst. Large, unpredictable jitter.
Polling (busy or fast task)	Fully deterministic timing, easiest to certify, no ISR.	Must poll at least every 160 µs during bursts (16 B FIFO). Busy-wait costs about 7% (S1) plus 6% (S2) of a core, starves lower tasks, and fails if a frame is late or a poll slips.

Option	Pros	Cons for this timing budget
DMA plus frame-end (idle-line) interrupt	One ISR per frame (300 per second, about 0.05% CPU). Bytes land with no CPU cost. Idle-line gives automatic frame resync.	Needs DMA and ring-buffer handling. ISR latency adds a small, bounded jitter.

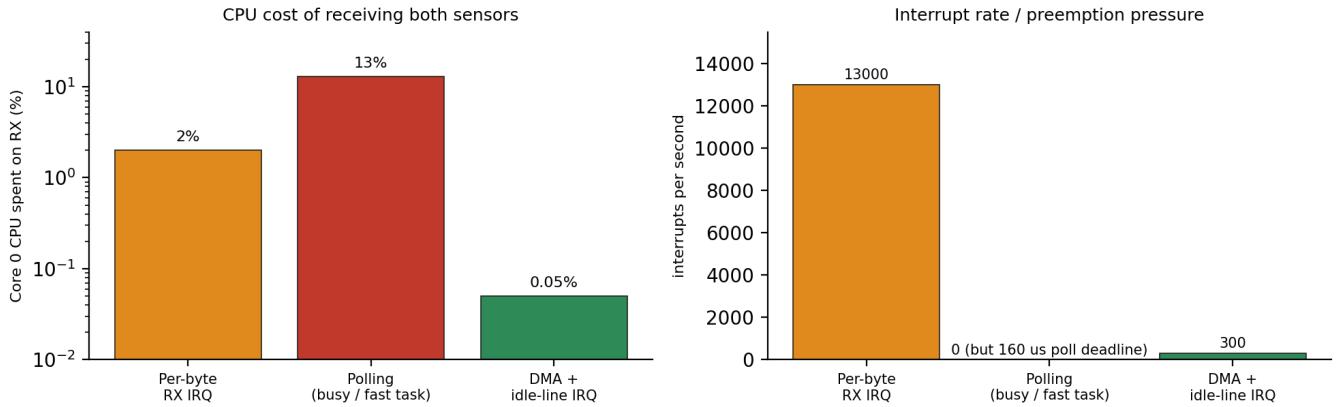


Figure 2. CPU cost and interrupt rate of the three receive strategies for both sensors.

- **Sensor 1 and Sensor 2 RX: DMA plus idle-line interrupt.** Arrival times are asynchronous and unknown to the MCU, so polling would waste a core. A single 5 µs ISR per frame defers all validation to T1.
- **Actuator TX (port 3): DMA, polled completion.** T2 starts the transfer (0.26 ms on the wire) and checks the previous transfer's done flag next cycle. Completion time is known and about 40 times shorter than the period, so an interrupt adds nothing.
- **Telemetry TX (port 4): DMA, polled completion**, for the same reason (0.64 ms transfer in a 10 ms period).
- **Rule applied:** interrupts where arrival is asynchronous (RX), polling where we control the timing (TX).

1.5 Timing and determinism analysis

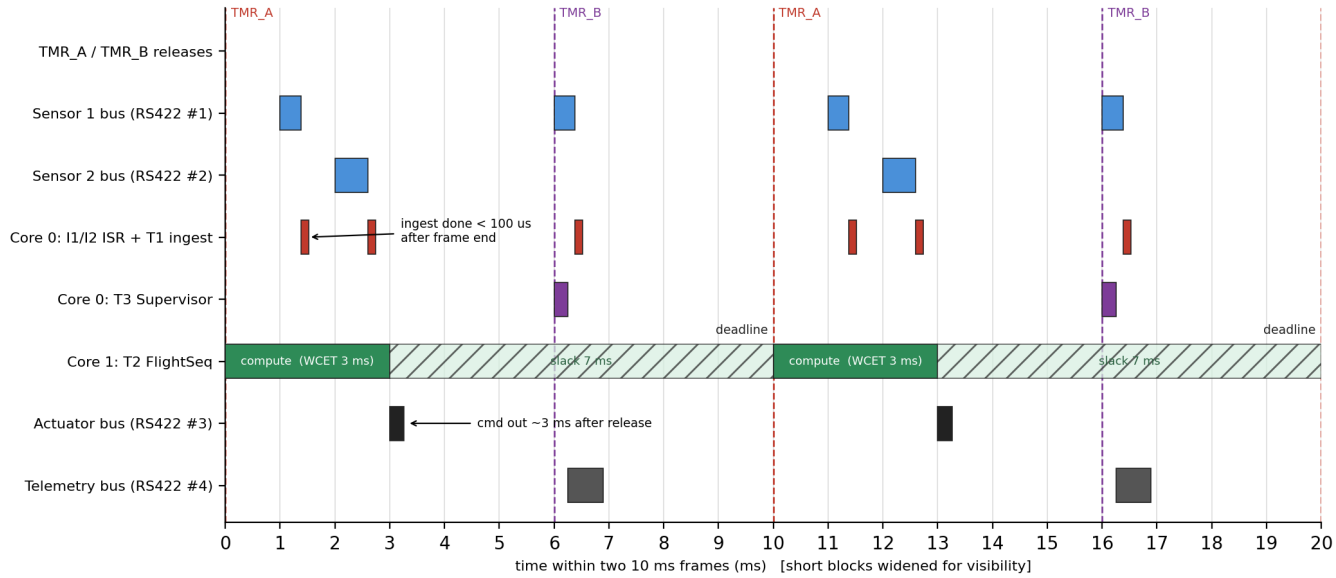


Figure 3. Two 10 ms frames across both cores and all four RS422 buses. Sensor phases are arbitrary; the design does not depend on them.

- **5 ms Sensor 1 deadline:** DMA captures the frame without the CPU. Ingest latency = ISR (5 µs) + at most one other ingest in progress (40 µs) + own work (40 µs), under 100 µs, about 2% of the period.
- **10 ms sequencer deadline:** T2 owns Core 1. R = WCET 3 ms + ISR 3 µs + 30% bus-contention allowance, about 3.9 ms against 10 ms, so 6 ms of slack.
- **Core 0 utilisation:** ISRs and ingest about 1.1%, supervisor 1.5%, slow task 20%, total about 23%. The slow task's response time is about 45 ms against 200 ms.
- **Data age at the sequencer** (worst case, unsynchronised sensors): Sensor 1 under 5.5 ms, Sensor 2 under 10.7 ms. Staleness limits are 7.5 ms and 15 ms (1.5 periods); exceeding them means hold plus flag.

- **Overrun detection:** counting notifications reveal missed releases, and any WCET breach is reported in telemetry and gates the watchdog.

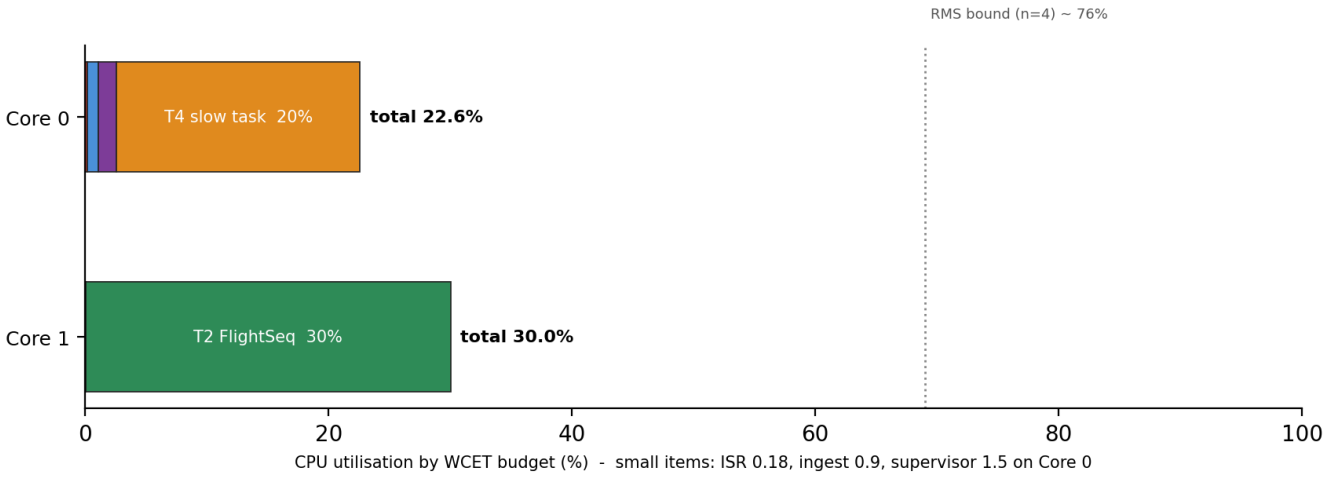


Figure 4. CPU utilisation by WCET budget. Core 1 is isolated from all asynchronous load.

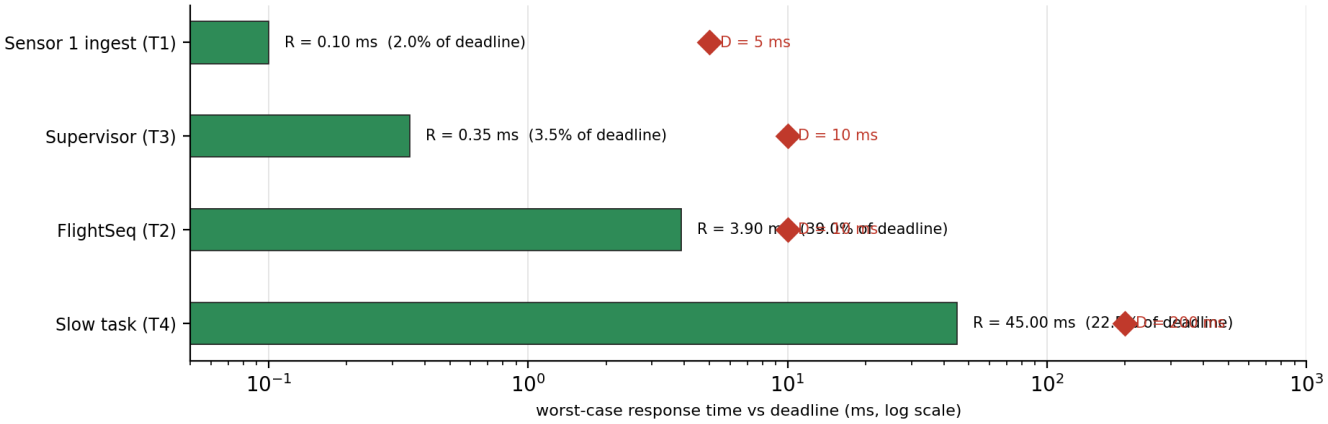


Figure 5. Worst-case response time against deadline for each periodic or event-driven activity.

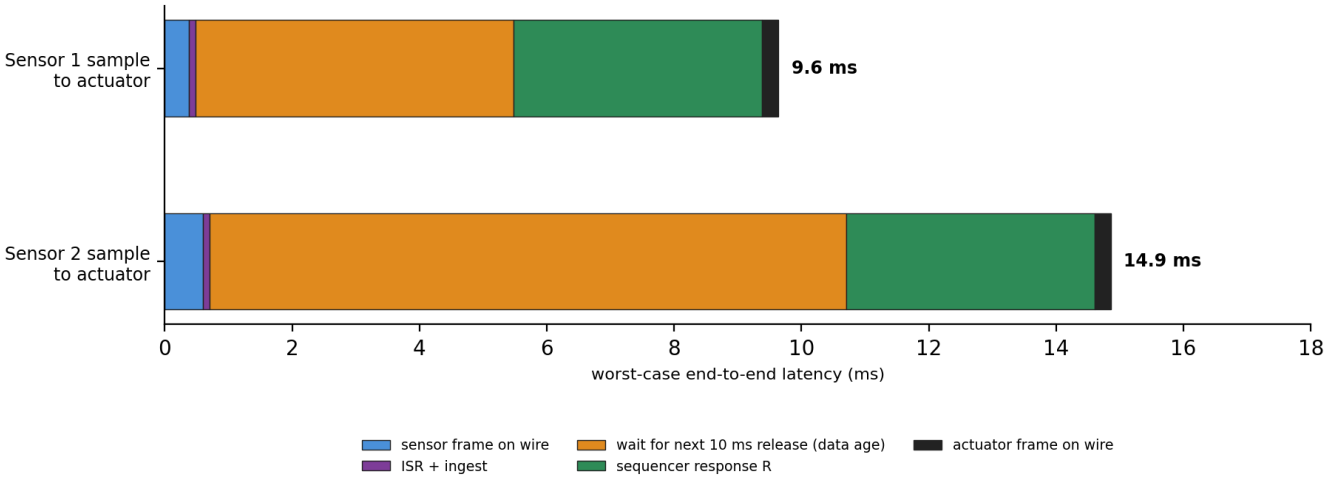


Figure 6. Worst-case sample-to-actuator latency budget for each sensor.

2. Synchronization and Concurrency

2.1 Shared resources and critical sections

Resource	Writer	Readers	Critical section
g_nav1 (32 B + timestamp)	T1 (Core 0)	T2 (Core 1), T4	Mailbox write / copy
g_nav2 (54 B + timestamp)	T1 (Core 0)	T2 (Core 1), T4	Mailbox write / copy
g_seqOut (16 B)	T2 (Core 1)	T4 (Core 0)	Mailbox write / copy
g_fb (16 B feedback loop)	T4 (Core 0)	T2 (Core 1)	Mailbox write / copy

Resource	Writer	Readers	Critical section
Health block	Each field has one writer	T3	None (atomic word access)
UART RX rings	DMA	I1 / I2, then T1	Producer-index handoff via ISR
Actuator and telemetry TX buffers	T2 / T3	DMA	Owned by the sender; busy flag checked first
g_frameId	I3	T2, I4	Single atomic word

2.2 Data protection

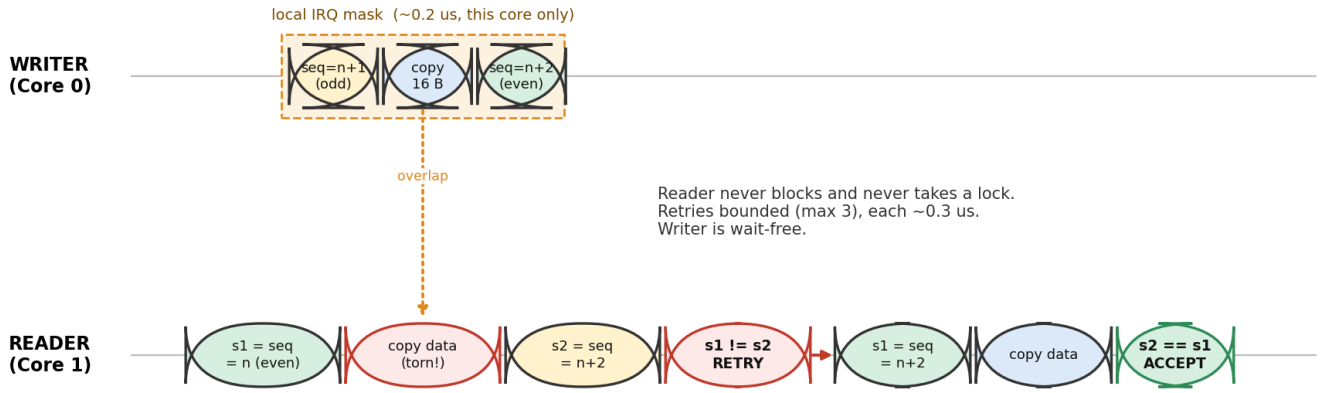


Figure 7. Seqlock mailbox protocol: the writer is wait-free, and a reader that overlaps a write detects it and retries.

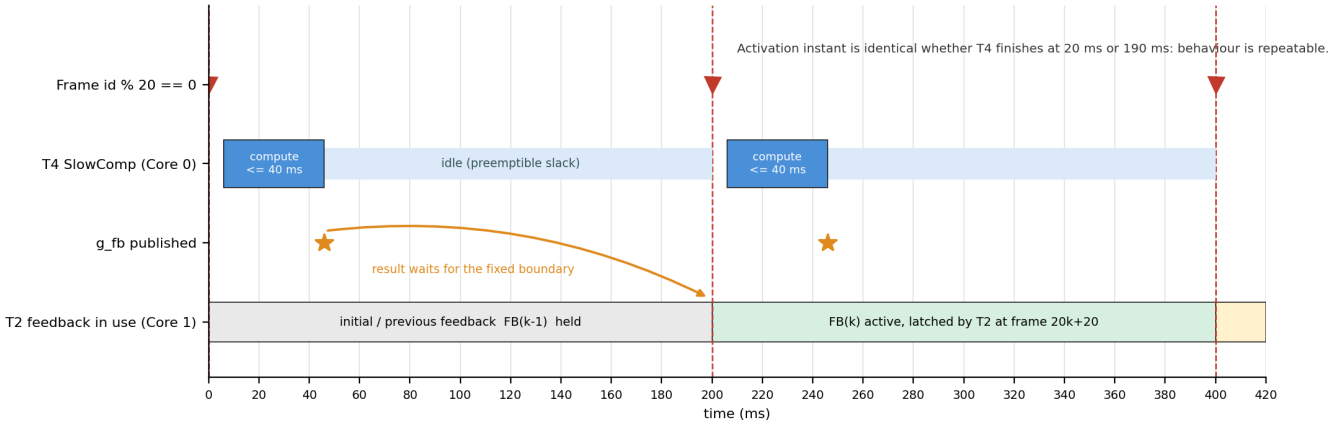


Figure 8. The 200 ms feedback loop: the result is activated at a fixed frame boundary regardless of when T4 finishes.

- **Seqlock mailboxes** carry a sequence counter (odd means write in progress). The writer takes a roughly 0.2 µs critical section with local-core interrupts masked, so it cannot be preempted mid-write. The reader copies, rechecks the counter and retries at most 3 times, so retry cost is bounded and tiny. The hard-real-time reader never blocks and never waits on a lock held by a low-priority task.
- **No race conditions:** single writer per object, release/acquire ordering (DMB on ARM), non-cacheable shared RAM and 64-byte alignment against false sharing. The local interrupt mask is used instead of the kernel's SMP critical section, because that takes a global spinlock both cores would contend on.
- **No deadlock:** no nested locks, no blocking call while any exclusion is held, and no lock shared between cores. The only exclusion is a local IRQ mask of bounded length.
- **No priority inversion:** the high-priority consumer (T2) never waits on a lower-priority producer. Kernel priority inheritance does not work across cores, which is a further reason to avoid mutexes here. Any mutex is confined to non-critical Core 0 code with priority inheritance enabled.
- **Different task frequencies:** latest-value semantics decouple the 5, 10 and 200 ms rates. Each consumer snapshots at the start of its cycle and works on the copy. Timestamps and frame IDs let tasks and the supervisor judge staleness.
- **Feedback loop integrity:** g_fb carries activate_frame, so T2 latches new feedback only on a 20-frame boundary. Output is deterministic, and T2 holds the previous value (and reports staleness) if the slow task is late.

3. Pseudocode (C-style, FreeRTOS-SMP-like API)

3.1 Common definitions

```
#define CORE0  (1u << 0)
#define CORE1  (1u << 1)
#define FRAME_US   10000u
#define SLOW_DIV   20u
#define N1_LEN 32
#define N2_LEN 54
#define ACT_LEN 20
#define FB_LEN 16
#define MBX_MAX_RETRY 3
#define NAV1_MAX_AGE_US 7500u
#define NAV2_MAX_AGE_US 15000u

typedef struct ALIGN64 {
    atomic_uint seq;            /* even = stable, odd = writing */
    uint64_t    ts_us;
    uint32_t    frame_id;
    uint8_t     data[64];
} Mbx_t;

static Mbx_t g_nav1, g_nav2, g_seqOut, g_fb;   /* one writer each */
static atomic_uint g_frameId;

typedef struct ALIGN64 {                       /* one writer per field */
    atomic_uint s1_last_ts, s1_frames, s1_crc_err, s1_seq_gap, s1_sync_err;
    atomic_uint s2_last_ts, s2_frames, s2_crc_err, s2_seq_gap, s2_sync_err;
    atomic_uint seq_hb, seq_exec_us, seq_exec_max_us, seq_missed_rel, act_busy_cnt;
    atomic_uint slow_hb, slow_exec_us, fb_stale_cnt;
} Health_t;
static Health_t g_health;

static void Mbx_Publish(Mbx_t *m, const void *src, size_t n, uint64_t ts, uint32_t fid)
{
    uint32_t irq = LocalIrqSave();            /* this core only, ~0.2 us */
    uint32_t s = atomic_load_explicit(&m->seq, memory_order_relaxed);
    atomic_store_explicit(&m->seq, s + 1, memory_order_relaxed);
    atomic_thread_fence(memory_order_release);
    memcpy(m->data, src, n);  m->ts_us = ts;  m->frame_id = fid;
    atomic_store_explicit(&m->seq, s + 2, memory_order_release);
    LocalIrqRestore(irq);
}

static bool Mbx_Read(const Mbx_t *m, void *dst, size_t n, uint64_t *ts, uint32_t *fid)
{
    for (int i = 0; i < MBX_MAX_RETRY; i++) {
        uint32_t s1 = atomic_load_explicit(&m->seq, memory_order_acquire);
        if (s1 & 1u) continue;
        memcpy(dst, m->data, n);  *ts = m->ts_us;  *fid = m->frame_id;
        atomic_thread_fence(memory_order_acquire);
        if (s1 == atomic_load_explicit(&m->seq, memory_order_relaxed)) return true;
    }
    return false;                             /* caller keeps last good copy */
}
```

3.2 System initialization (tasks, IPC, core affinity)

```
static TaskHandle_t hIngest, hSeq, hSup, hSlow;

int main(void)
{
    BSP_ClockInit();
    BSP_MpuMapSharedNonCacheable();
    HwTimeBase_Init();                        /* shared 1 us counter */
    memset(&g_health, 0, sizeof g_health);

    Rs422_Init(UART_S1,  1000000);  Dma_RxCircular(UART_S1,  g_s1Ring,  RING_S1);
    Rs422_Init(UART_S2,  1000000);  Dma_RxCircular(UART_S2,  g_s2Ring,  RING_S2);
    Rs422_Init(UART_ACT, 1000000);  Dma_TxInit(UART_ACT);
    Rs422_Init(UART_TLM, 1000000);  Dma_TxInit(UART_TLM);

    /* Statically allocated tasks pinned to cores */
    xTaskCreateAffinitySet(SensorIngestTask, TASK_NAME_ING, STK_ING, NULL, 5, CORE0, &hIngest);
    xTaskCreateAffinitySet(SupervisorTask,   TASK_NAME_SUP, STK_SUP, NULL, 4, CORE0, &hSup);
    xTaskCreateAffinitySet(SlowCompTask,     TASK_NAME_SLW, STK_SLW, NULL, 2, CORE0, &hSlow);
    xTaskCreateAffinitySet(FlightSeqTask,    TASK_NAME_SEQ, STK_SEQ, NULL, 5, CORE1, &hSeq);

    /* Route interrupts to cores */
    Irq_Route(IRQ_UART_S1, CORE0, IRQ_LVL1);
    Irq_Route(IRQ_UART_S2, CORE0, IRQ_LVL1);
    Irq_Route(IRQ_TMR_B,   CORE0, IRQ_LVL2);
    Irq_Route(IRQ_TMR_A,   CORE1, IRQ_LVL2);

    vTaskStartScheduler();                    /* both cores come up */
}

/* Runs once after the scheduler is live on both cores: tasks and handles exist */
void vApplicationStartupHook(void)
```

```
{
    Uart_EnableIdleIrq(UART_S1);  Uart_EnableIdleIrq(UART_S2);
    HwTimer_StartPeriodic(TMR_A, FRAME_US, 0);       /* sequencer at phase 0     */
    HwTimer_StartPeriodic(TMR_B, FRAME_US, 6000);    /* supervisor/slow at +6 ms */
}
```

3.3 Peripheral reception (DMA plus idle-line ISR, deferred processing)

```
void UART_S1_IRQHandler(void)                /* Core 0 */
{
    uint32_t st = Uart_Status(UART_S1);
    if (st & UART_IDLE) {
        Uart_ClearIdle(UART_S1);
        g_s1.end_idx = Dma_WriteIndex(DMA_S1);
        g_s1.isr_ts  = HwTime_us();
        BaseType_t hp = pdFALSE;
        xTaskNotifyFromISR(hIngest, EVT_S1, eSetBits, &hp);
        portYIELD_FROM_ISR(hp);
    }
    if (st & (UART_ORE | UART_FE | UART_NE)) {
        Uart_ClearErr(UART_S1);
        atomic_fetch_add(&g_health.s1_sync_err, 1);
    }
}
/* UART_S2_IRQHandler is identical, using g_s2 and EVT_S2 */

static void IngestOne(RxCtx_t *c, Mbx_t *mbx, size_t len,
                      atomic_uint *last_ts, atomic_uint *frames,
                      atomic_uint *crc_err, atomic_uint *seq_gap, atomic_uint *sync_err)
{
    Frame_t f;
    if (!Ring_ExtractFrame(c, &f, len))      { atomic_fetch_add(sync_err, 1); return; }
    if (Crc16(f.raw, f.n - 2) != f.crc)      { atomic_fetch_add(crc_err, 1);  return; }
    uint8_t gap = (uint8_t)(f.seq - c->last_seq - 1);
    if (gap) atomic_fetch_add(seq_gap, gap);
    c->last_seq = f.seq;
    Mbx_Publish(mbx, f.payload, len, c->isr_ts, 0);
    atomic_store(last_ts, (uint32_t)c->isr_ts);
    atomic_fetch_add(frames, 1);
}

void SensorIngestTask(void *arg)
{
    for (;;) {
        uint32_t ev = 0;
        xTaskNotifyWait(0, 0xFFFFFFFFu, &ev, portMAX_DELAY);
        if (ev & EVT_S1) IngestOne(&g_s1, &g_nav1, N1_LEN, &g_health.s1_last_ts,
                                   &g_health.s1_frames, &g_health.s1_crc_err,
                                   &g_health.s1_seq_gap, &g_health.s1_sync_err);
        if (ev & EVT_S2) IngestOne(&g_s2, &g_nav2, N2_LEN, &g_health.s2_last_ts,
                                   &g_health.s2_frames, &g_health.s2_crc_err,
                                   &g_health.s2_seq_gap, &g_health.s2_sync_err);
    }
}
```

3.4 Timer ISRs and the 10 ms Flight Sequencer task

```
void TMR_A_IRQHandler(void)                  /* Core 1, phase 0 */
{
    HwTimer_ClearIrq(TMR_A);
    static uint32_t cnt;
    atomic_store(&g_frameId, cnt++);
    BaseType_t hp = pdFALSE;
    vTaskNotifyGiveFromISR(hSeq, &hp);       /* counting: exposes missed releases */
    portYIELD_FROM_ISR(hp);
}

void TMR_B_IRQHandler(void)                  /* Core 0, phase +6 ms */
{
    HwTimer_ClearIrq(TMR_B);
    BaseType_t hp = pdFALSE;
    vTaskNotifyGiveFromISR(hSup, &hp);
    if ((atomic_load(&g_frameId) % SLOW_DIV) == 0)
        vTaskNotifyGiveFromISR(hSlow, &hp);
    portYIELD_FROM_ISR(hp);
}

void FlightSeqTask(void *arg)                /* Core 1, prio 5 */
{
    Nav1_t n1 = {0};  Nav2_t n2 = {0};  FbPayload_t fb = {0};  FbMsg_t fbTmp;
    FSeqState_t st;   FSeq_Init(&st);
    ActCmd_t cmd;     SeqOut16_t out16;
    uint32_t actSeq = 0, fbCycle = 0;

    for (;;) {
        uint32_t rel = ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
        if (rel > 1) atomic_fetch_add(&g_health.seq_missed_rel, rel - 1);
        uint64_t t0 = HwTime_us();
        uint32_t frame = atomic_load(&g_frameId);

        /* 1. Ingest latest nav data (wait-free snapshot, keep last good on failure) */
        uint64_t ts1, ts2;  uint32_t d;  Nav1_t t1;  Nav2_t t2;
```

```
        if (Mbx_Read(&g_nav1, &t1, N1_LEN, &ts1, &d)) n1 = t1;
        if (Mbx_Read(&g_nav2, &t2, N2_LEN, &ts2, &d)) n2 = t2;
        bool v1 = (t0 - ts1) <= NAV1_MAX_AGE_US;
        bool v2 = (t0 - ts2) <= NAV2_MAX_AGE_US;

        /* 2. Feedback loop: latch the slow-task result only on a 20-frame boundary */
        if ((frame % SLOW_DIV) == 0) {
            uint64_t tsf;
            if (Mbx_Read(&g_fb, &fbTmp, FB_LEN, &tsf, &d) &&
                fbTmp.cycle_id != fbCycle && fbTmp.activate_frame <= frame) {
                fb = fbTmp.payload;  fbCycle = fbTmp.cycle_id;
            } else {
                atomic_fetch_add(&g_health.fb_stale_cnt, 1);   /* hold previous */
            }
        }

        /* 3. Flight computation (placeholder) */
        ComputeFlightActuation(&st, &n1, &n2, v1, v2, &fb, &cmd, &out16);

        /* 4. Publish the 16 B output for the slow task */
        Mbx_Publish(&g_seqOut, &out16, 16, t0, frame);

        /* 5. Transmit the 20 B actuator command (DMA, non-blocking) */
        if (!Dma_TxBusy(UART_ACT)) {
            Frame_Build(g_actTx, SYNC_ACT, (uint8_t)actSeq++, &cmd, ACT_LEN);
            Dma_TxStart(UART_ACT, g_actTx, ACT_FRAME_LEN);
        } else {
            atomic_fetch_add(&g_health.act_busy_cnt, 1);
        }

        /* 6. Health */
        uint32_t dt = (uint32_t)(HwTime_us() - t0);
        atomic_store(&g_health.seq_exec_us, dt);
        if (dt > atomic_load(&g_health.seq_exec_max_us))
            atomic_store(&g_health.seq_exec_max_us, dt);
        atomic_fetch_add(&g_health.seq_hb, 1);
    }
}
```

3.5 The 200 ms Slow Computation task

```
void SlowCompTask(void *arg)                   /* Core 0, prio 2 */
{
    Nav1_t n1;  Nav2_t n2;  SeqOut16_t so;  FbMsg_t fb;  uint32_t cycle = 0;

    for (;;) {
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);          /* from TMR_B_ISR, every 20th frame */
        uint64_t t0 = HwTime_us();

        /* Snapshot all inputs at the start of the cycle, then work on local copies */
        uint64_t ts;  uint32_t fidNav, fidSeq;
        Mbx_Read(&g_nav1,   &n1, N1_LEN, &ts, &fidNav);
        Mbx_Read(&g_nav2,   &n2, N2_LEN, &ts, &fidNav);
        Mbx_Read(&g_seqOut, &so, 16,     &ts, &fidSeq);

        CalculateSlowDynamics(&n1, &n2, &so, &fb.payload);

        fb.cycle_id       = ++cycle;
        fb.activate_frame = ((fidSeq / SLOW_DIV) + 1) * SLOW_DIV;   /* next boundary */
        Mbx_Publish(&g_fb, &fb, FB_LEN, HwTime_us(), fidSeq);

        atomic_store(&g_health.slow_exec_us, (uint32_t)(HwTime_us() - t0));
        atomic_fetch_add(&g_health.slow_hb, 1);
    }
}
```

3.6 The 10 ms Supervisory task

```
void SupervisorTask(void *arg)                 /* Core 0, prio 4 */
{
    uint32_t lastSeqHb = 0, lastSlowHb = 0, slowSilent = 0;
    TlmPacket_t pkt;  uint8_t tlmSeq = 0;

    for (;;) {
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);          /* frame start + 6 ms */
        uint32_t now = (uint32_t)HwTime_us();
        uint32_t frame = atomic_load(&g_frameId);
        uint32_t flags = 0;

        /* Sensor interfaces: link alive if the latest frame is recent */
        if (now - atomic_load(&g_health.s1_last_ts) > NAV1_MAX_AGE_US) flags |= F_S1_TIMEOUT;
        if (now - atomic_load(&g_health.s2_last_ts) > NAV2_MAX_AGE_US) flags |= F_S2_TIMEOUT;
        if (atomic_load(&g_health.s1_crc_err) || atomic_load(&g_health.s2_crc_err)) flags |= F_NAV_CRC;

        /* Flight sequencer: heartbeat must advance every frame */
        uint32_t hb = atomic_load(&g_health.seq_hb);
        if (hb == lastSeqHb) flags |= F_SEQ_STALLED;
        lastSeqHb = hb;
        if (atomic_load(&g_health.seq_exec_max_us) > SEQ_BUDGET_US) flags |= F_SEQ_OVERRUN;
        if (atomic_load(&g_health.seq_missed_rel)) flags |= F_SEQ_MISSED;

        /* Actuator interface */
```

```
        if (atomic_load(&g_health.act_busy_cnt)) flags |= F_ACT_TX;

        /* Slow task: heartbeat must advance within one 200 ms window (20 frames) */
        if ((frame % SLOW_DIV) == 0) {
            uint32_t sh = atomic_load(&g_health.slow_hb);
            slowSilent = (sh == lastSlowHb) ? slowSilent + 1 : 0;
            lastSlowHb = sh;
        }
        if (slowSilent >= 2) flags |= F_SLOW_STALLED;
        if (atomic_load(&g_health.fb_stale_cnt)) flags |= F_FB_STALE;

        /* Build and send the telemetry packet (DMA, non-blocking) */
        Tlm_Build(&pkt, tlmSeq++, frame, flags, &g_health);
        if (!Dma_TxBusy(UART_TLM)) Dma_TxStart(UART_TLM, &pkt, sizeof pkt);
        else                       flags |= F_TLM_TX;

        /* Watchdog: kicked only when every monitored element is healthy */
        if ((flags & F_CRITICAL_MASK) == 0) Wdt_Kick();
    }
}
```

4. Evaluation-criteria traceability

Criterion	How the design meets it	Where
Real-time determinism	Isolated hard-RT core, hardware-timer release, DMA RX, wait-free writers, bounded readers, response-time analysis with 6 ms slack, missed-release and overrun detection.	1.2, 1.5, Fig. 3 to 6
RTOS fundamentals	Notifications for events, counting notifications for periodic release, seqlock mailboxes for state, queue only for same-core rare events, mutexes avoided on the hard path.	1.3, 2.2, 3
Multi-core competency	Core 1 dedicated to the sequencer; four single-writer mailboxes as the only boundary; memory barriers, non-cacheable shared RAM, false-sharing control; local IRQ mask instead of the SMP kernel lock.	1.2, 2.1, 2.2, Fig. 1, 7
Engineering judgment	Nine documented assumptions with WCET budgets, frame formats and wire times; failure policy for stale data; fixed-boundary feedback activation.	0, 1.5, Fig. 8
Design justification	Quantified interrupt vs polling comparison (CPU cost, interrupt rate, FIFO deadline) with a per-port decision; lock-free vs mutex vs queue rationale.	1.4, 2.2, Fig. 2